

Exercises — Dynamic Programming

Lecture 2/4 — Reinforcement Learning

CentraleSupélec

Written Exercises — Pen and Paper

Topics covered

- **Bellman equations:** writing $v_\pi(s)$ and $v_*(s)$ from a given MDP.
- **Policy evaluation:** iterating the update $v(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$ until convergence.
- **Policy improvement:** computing the greedy policy $\pi'(s) = \arg \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$.
- **Policy iteration:** evaluation $\rightarrow$ improvement cycles until stability.
- **Value iteration:** iterating the Bellman optimality equation $v(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r + \gamma v(s')]$.

Structure

The exercises are organized around two model environments:

1. **3×3 grid** — terminal state top-left, reward -1 per step, $\gamma = 1$. Used to practice policy evaluation and improvement (Exercises 1–2), then value iteration on the same environment (Exercise 3).
2. **Gambler's problem** — discretized capital, integer stakes, fair coin flips. Ideal for value iteration and studying non-unique optimal policies (Exercise 4).

Exercise 1 — Policy improvement theorem on a 3×3 gridworld

Consider a 3×3 gridworld with:

- States: $S = \{1, 2, \dots, 8\}$ (numbered row by row), plus a terminal state **T** top-left.
- Actions: $A = \{\text{up, down, left, right}\}$, deterministic, actions that would leave the grid keep the agent in place.
- Reward: $r(s, a, s') = -1$ for every s, a, s' (except the terminal state).
- Discount factor: $\gamma = 1$ (episodic task).

Part A. Policy evaluation

Compute $v^{\pi_0}(s)$ for all states, where π_0 is the **equiprobable random** policy ($\pi_0(a|s) = 0.25$ for all a, s).

- Write the Bellman equation for $v^{\pi_0}(s)$.
- By symmetry, states group together. Exploit this symmetry.
- Solve the resulting linear system (or iterate to convergence).

Part B. Action-value function (q^{π_0})

Compute $q^{\pi_0}(s, a)$ for each state s and each action a (at least for a few key states).

Recall: $q^{\pi_0}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v^{\pi_0}(s')]$

Part C. Testing the improvement theorem

The **policy improvement theorem** says that if you greedify the policy with respect to v^{π_0} :

$$\pi'(s) = \arg \max_a q^{\pi_0}(s, a)$$

then $v^{\pi'}(s) \geq v^{\pi_0}(s)$ for all s .

- Compute the greedy policy π' .
- Compare it with π_0 . Are they different?
- *Optional:* Evaluate $v^{\pi'}$ to check that $v^{\pi'}(s) \geq v^{\pi_0}(s)$.

Exercise 2 — Proof of the improvement theorem (theory bonus)

Formally prove the **policy improvement theorem**:

If $q^\pi(s, \pi'(s)) \geq v^\pi(s)$ for all $s \in S$, then $v^{\pi'}(s) \geq v^\pi(s)$ for all s .

Hint: Start by showing that $v^{\pi'}(s) \geq q^\pi(s, \pi'(s))$. Then use the hypothesis and one iteration of the Bellman equation.

Exercise 3 — Value iteration on the 3×3 grid

We reuse exactly the environment of Exercise 1 (states $\{1, \dots, 8\}$ numbered row by row, terminal T top-left, $r = -1$ per step, $\gamma = 1$), but now solve the optimality equation directly instead of going through evaluation + improvement.

Start from $V_0(s) = 0$ for every non-terminal state.

- Compute $V_1(s)$ for all states (a single sweep, $V_1(s) = \max_a [-1 + V_0(s')]$). What do you notice: are all 8 values equal? Why doesn't the grid's structure matter yet at this stage?
- Compute $V_2(s)$ for all states. Two different values should appear — which ones, and for which states?
- Without computing further:* what will $V_k(s)$ converge to as $k \rightarrow \infty$? (Hint: relate $V_k(s)$ to the number of steps needed to reach T from s , and compare to what you already observe in (a)–(b).) Give $V_*(8)$ (the state farthest from T).
- Deduce an optimal policy π_* for each state. At least two states admit **several** tied optimal actions — which ones, and why? Connect this to the lecture's remark on the non-uniqueness of π_* (slide “Example 3.8”).

Exercise 4 — Value iteration — gambler’s problem (reduced version)

A gambler bets on a sequence of **fair** coin flips ($p_h = 0.5$). They start with capital $s \in \{1, 2, 3\}$ and win as soon as they reach 4, and lose everything if they drop to 0. On each flip they stake an integer amount $a \in \{1, \dots, \min(s, 4 - s)\}$: heads $\Rightarrow$ capital $s + a$, tails $\Rightarrow$ capital $s - a$. Reward is zero everywhere except +1 on reaching 4. $\gamma = 1$.

- (a) Write the Bellman optimality equation for $V(s)$, $s \in \{1, 2, 3\}$ (set $V(0) = 0$, $V(4) = 0$ — the +1 reward is already counted on the transition that reaches 4).
- (b) Check that $V(s) = s/4$ solves this equation for **every** possible stake a at each state (not just the optimal stake).
- (c) What does this imply about the uniqueness of the optimal policy when $p_h = 0.5$? How many distinct optimal policies exist in that case?
- (d) (No computation) How does $V(s)$ qualitatively deform when $p_h < 0.5$? when $p_h > 0.5$? (cf. Sutton & Barto, example 4.3, for the sawtooth curves obtained when $p_h \neq 0.5$.)

Exercise 5 — Action values in Sutton & Barto’s own gridworld (classic, Ch. 4, Example 4.1)

Recall the *real* gridworld of Example 4.1: 4×4 , states $0, \dots, 15$ numbered row by row, states 0 and 15 terminal, $r = -1$ per step, equiprobable random policy π_0 . This is exactly the `GridworldEnv` you will use in the code section, and its converged v^{π_0} is already given in lecture (slide “Policy Evaluation — Value Function Evolution”):

$$v^{\pi_0} = [0, -14, -20, -22, -14, -18, -20, -20, -20, -20, -18, -14, -22, -20, -14, 0]$$

- (a) Using this table, compute $q^{\pi_0}(11, \text{down})$.
- (b) Compute $q^{\pi_0}(7, \text{down})$.
- (c) In one sentence, why is $q^{\pi_0}(11, \text{down})$ so much less negative than $q^{\pi_0}(7, \text{down})$, even though both take the same action?

Exercise 6 — Fixing policy iteration’s convergence bug (classic, Ch. 4)

The textbook’s policy iteration pseudocode can, in principle, fail to terminate if the policy keeps switching between two or more equally good policies at the “policy stable?” check.

- (a) Explain concretely how this could happen (hint: think about ties in the arg max step of policy improvement).
- (b) Propose a one-line fix to the stopping condition that guarantees termination regardless of how ties are broken.

Exercise 7 — The gambler’s curious optimal policy (classic, Ch. 4, full-scale problem)

In the **full-scale** gambler’s problem (capital 1 to 99, goal 100 — not the reduced $\{1, 2, 3\}$ version of Exercise 4), the optimal policy has a well-known odd shape: at capital 50 the optimal bet is to go **all-in** (stake 50), but at capital 51 it is **not** (a much smaller stake is optimal).

- (a) Give an intuitive argument for why staking everything is optimal exactly at capital 50.
- (b) Why does this no longer hold at capital 51?

Code Exercises — Implementation

Topics covered

- **Representing an MDP**: an `Environment` class or Gym environments.
- **Implementing policy evaluation**: nested loops, ϵ -convergence.
- **Implementing policy improvement**: computing the greedy argmax action.
- **Policy iteration**: orchestrating both steps, detecting stability.
- **Value iteration**: in-place vs. copy updates, handling terminal states.
- **Visualization**: value heatmaps, direction arrows, trajectory display.

Notebook structure

Each notebook follows a unified template:

1. **Imports and setup** — dependencies, environment definition.
2. **Theory recap** — equations and algorithm in Markdown.
3. **Step-by-step implementation** — functions or a class for the algorithm.
4. **Run and display** — convergence plots, final visualization.
5. **Open questions** — variations (γ sweeps, larger environments, different stopping conditions).

Environments used

- **Gridworld** — 4×4 or 5×5 grid, terminal states, -1 reward per step.
- **Variations** — obstacles, mixed rewards, etc. depending on the exercise.

Expectations

- Code should be **readable**: named variables, comments on loops.
- Results should be **plotted**: convergence graphs (iteration count vs. error), heatmaps of final values.
- Answers to questions should be argued, not just a screenshot.

The 5 exercises below are coded in the companion notebook `tutoriel.ipynb` (same folder), which already provides the `GridworldEnv` environment (4×4 grid, 2 terminal states — `tutorials/lib/`, the same one used for the figure in the “Policy Evaluation — Gridworld Example” slide) and the plotting functions. Each exercise corresponds to a cell with a function signature to fill in (`raise NotImplementedError`) and a test cell for self-checking.

Exercise 8 — Iterative policy evaluation (code)

Implement `policy_eval(policy, env, gamma, theta)`: successive sweeps of $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s, \gamma V(s'))$ until $\max_s |V_{k+1}(s) - V_k(s)| < \theta$. Test it with the equiprobable random policy on `GridworldEnv`:

you should recover the values $v^\pi(s) \in \{-14, \dots, -22, 0\}$ from the “Policy Evaluation — Value Function Evolution” slide in lecture — this is your regression test.

Exercise 9 — Policy improvement (code)

Implement `policy_improvement(V, env, gamma)`, which returns, for each state, the greedy action(s) (or tied actions) with respect to V . Applied to the v^π found in the previous exercise, the resulting policy already reaches v_* in a single pass — why (hint: this is a fact specific to this particular grid, not a general property)? Careful: **“optimal” here means “reaches v_* ”**, not “matches term by term” the policy from Exercise 7 — the two can break a tied state differently without either being better than the other.

Exercise 10 — Policy iteration (code)

Assemble `policy_iteration(env, gamma)`: alternate evaluation (Exercise 8) and improvement (Exercise 9) until the policy stops changing. Display the number of evaluation/improvement cycles needed, and the final policy as arrows on the grid.

Exercise 11 — Value iteration (code)

Implement `value_iteration(env, gamma, theta)`: sweeps of $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$, without ever separating evaluation and improvement. Careful, unlike policy evaluation (Exercise 8, which converges to v^π of the *random* policy), here V converges directly to v_* : check that it matches the v_* found by policy iteration (Exercise 10), and that the final greedy policy **reaches the same value** (not necessarily the same action-by-action choice on tied states — compare values, not action indices).

Exercise 12 — Policy iteration vs. value iteration (bonus)

On the same axes, plot $\max_s |V_k(s) - V_*(s)|$ against the number of **sweeps** (not cycles) for the two algorithms of Exercises 10 and 11. Which converges in fewer sweeps on this grid? Repeat on an 8×8 grid (`GridworldEnv(shape=[8,8])`): does the gap widen? *Open question, no single expected answer — justify with your plots.*

Notation & formulae

- **Policy evaluation** (fixed π , iterate to convergence):

$$v(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v(s')] \quad \forall s$$

- **Policy improvement** (greedy w.r.t. v):

$$\pi'(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v(s')]$$

- **Policy improvement theorem**: if $q^\pi(s, \pi'(s)) \geq v^\pi(s)$ for all s , then $v^{\pi'}(s) \geq v^\pi(s)$ for all s .
- **Policy iteration**: repeat evaluation $\rightarrow$ improvement until $\pi' = \pi$ (guaranteed to converge to π_* on a finite MDP).
- **Value iteration** (fold evaluation and improvement into one update):

$$v(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v(s')] \quad \forall s$$

then extract $\pi_*(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')]$ once v has converged.

- **Generalized Policy Iteration (GPI)**: any interleaving of (possibly partial) evaluation and improvement steps converges to (v_*, π_*) — policy iteration and value iteration are the two extremes of the same idea.
- **Stopping criterion**: stop sweeping once $\max_s |v_{k+1}(s) - v_k(s)| < \theta$ for a small θ .